写出下列程序的运行结果

题目

```
                              int main()
{   const char *c[]={"John learn C++ language",
                     "Be well!", "You", "Not very"};
    const char **p[]={c+3, c+2, c+1, c};
    const char ***pp=p;
    cout << (**++pp);
    cout << (*--*++pp+4);
    cout << (*pp[-2]+3);
    cout << (pp[-1][-1]+2);
    cout << endl;
    return 0;
}
```

三条定义语句

c 是 4 个指向字符串常量的指针；p 是 4 个指向 c 中元素的指针；pp 指向 p 的首元素。

四条输出语句

每条语句按“先定值、再取值”逐算子拆步，每步一页，观察 pp、p、c 三级指针的移动方向与解引用层数。

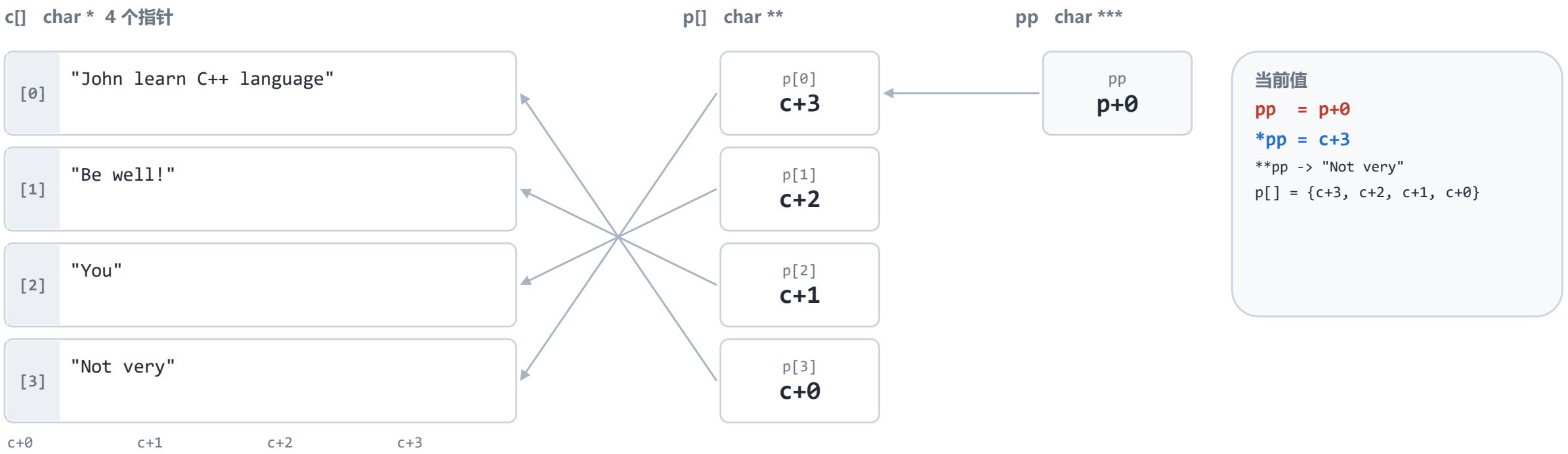
关键约定

p[i] 里存的是“指向第几个字符串”的地址，记作 c+k；pp 里存的是 p+k。负下标合法，指针可以往回退。

三条定义语句结束后的内存布局

初始态

```
const char *c[] = { "John learn C++ language", "Be well!", "You", "Not very" };   const char **p[] = { c+3, c+2, c+1, c };   const char ***pp = p;
```



① c[0..3] 各指向一个字符串常量的首字符；c+i 是“第 i 个元素的地址”，所以 p 里存 c+k 表示“指向第 k 个字符串”。

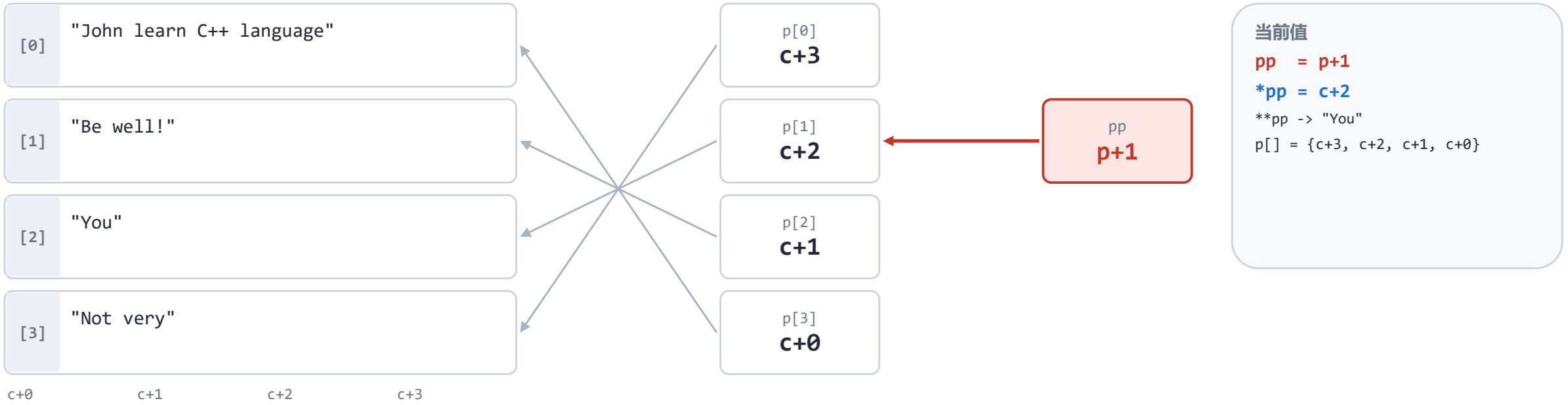
② p 的初值 {c+3, c+2, c+1, c} 是倒序的：p[0]→c[3]、p[1]→c[2]、p[2]→c[1]、p[3]→c[0]，箭头因此交叉。

③ pp = p，即指向 p 的首元素，故 pp == p == &p[0]，*pp 就是 p[0] 的值。

① 先算 ++pp：pp 指向 p[1]

```
cout << (**++pp);
```

累计输出：（空）



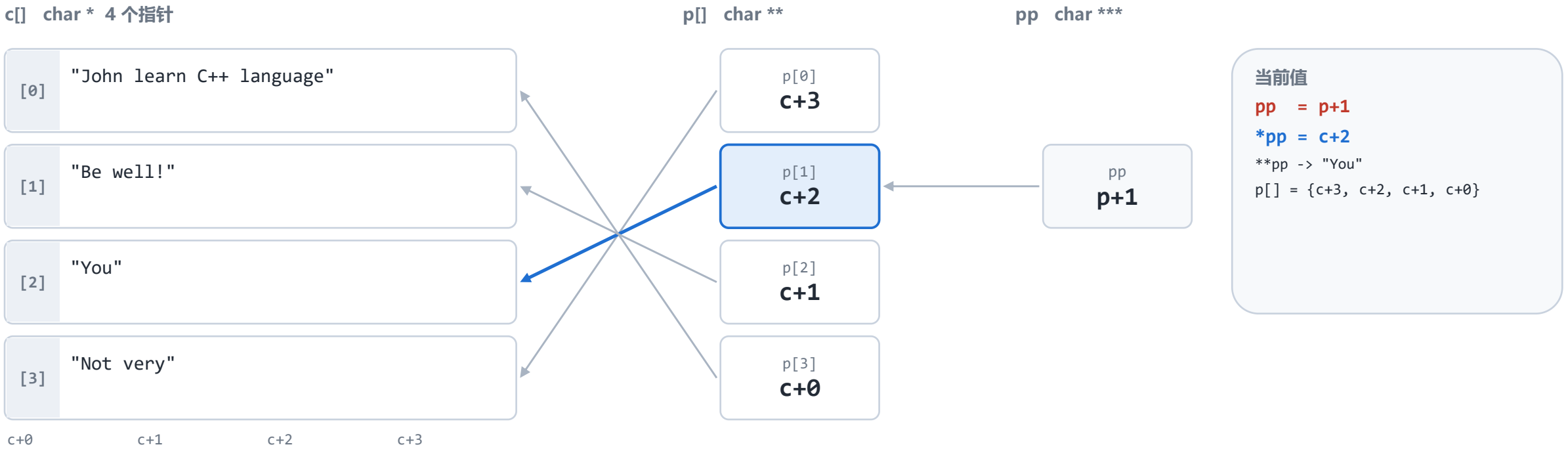
++pp 是前置自增：pp 自身加 1，由 p+0 变为 p+1，整个表达式的值也是 p+1。

此步只改 pp，p 数组与 c 数组均未变动。

② 再算 *pp：取出 p[1] 的值 c+2

```
cout << (**++pp);
```

累计输出：（空）



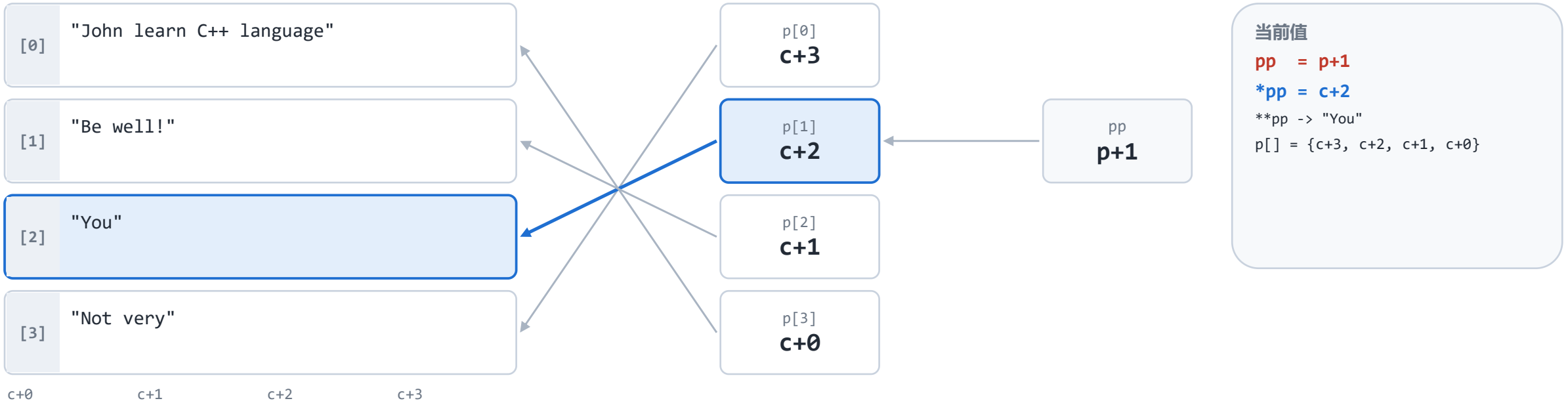
*（pp 的新值）= *(p+1) = p[1]，读出 p 数组第 1 个元素里存的地址：c+2。

此刻结果仍是一个 char ** 类型的地址，尚未取出任何字符，所以还没有输出。

③ 外层 * 解引用：取出字符串 You

```
cout << (**++pp);
```

累计输出：You



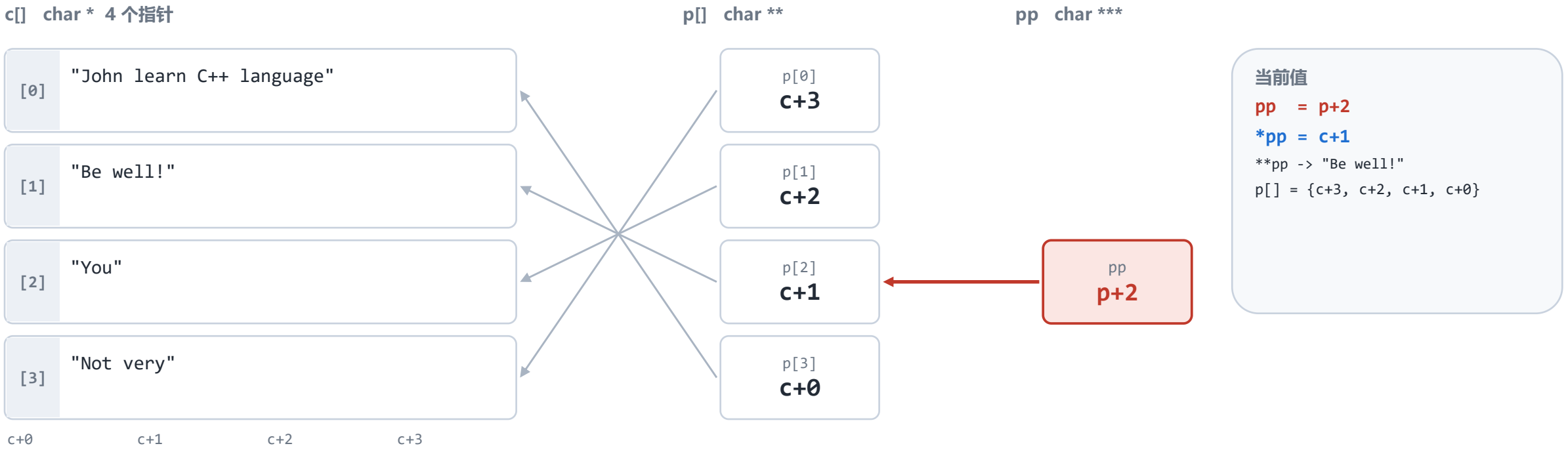
再解一层：**(p+1) = *(c+2)，即 c[2] 所指字符串 "You" 的首地址。

cout 输出该字符串：You。第 1 条语句共输出 3 个字符。

① ++pp：pp 再进一格，指向 p[2]

```
cout << (*--*++pp+4);
```

累计输出：You



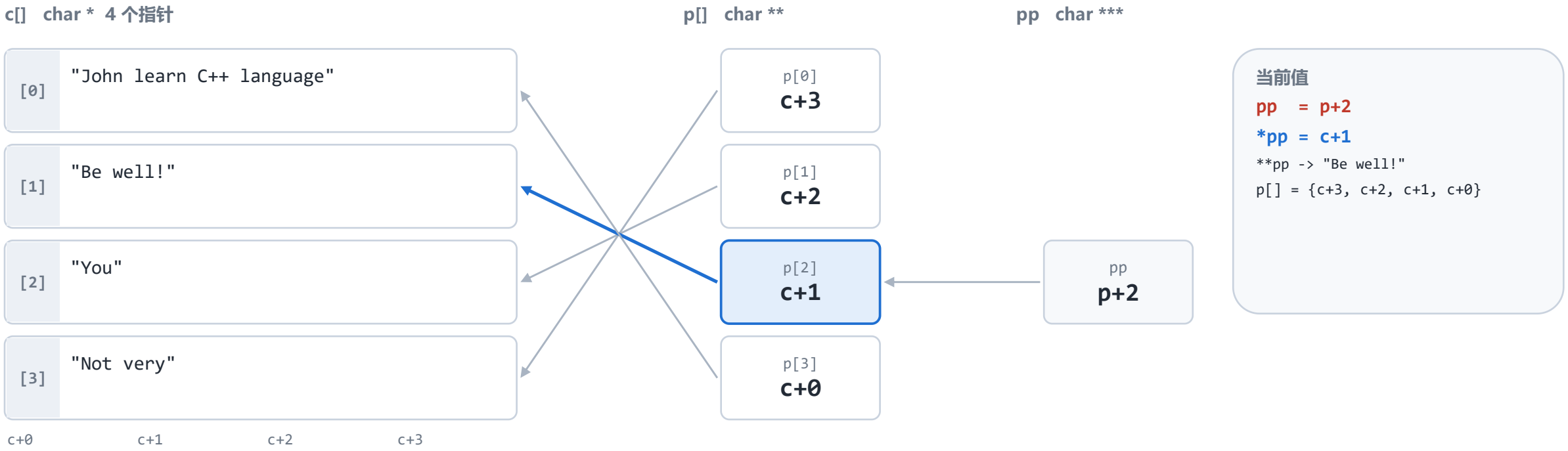
按优先级从内向外拆：先做最内层的 ++pp。pp 由 p+1 变为 p+2。

这一步仍然只动 pp，p[2] 里还是 c+1。

② *++pp：取出 p[2] 的值 c+1

```
cout << (*--*++pp+4);
```

累计输出：You



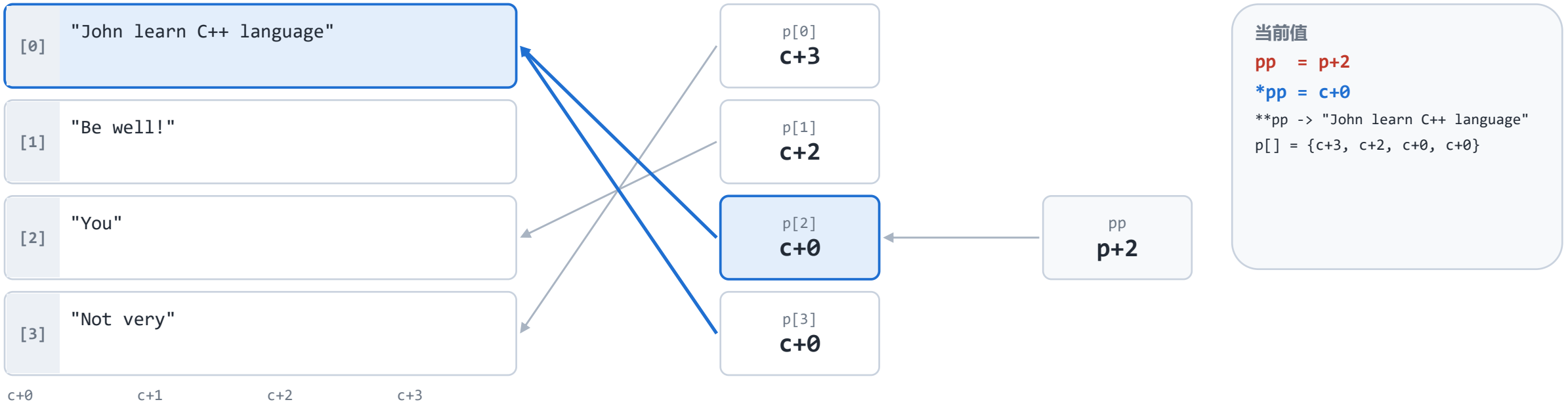
*（p+2）= p[2]，其值为 c+1，即指向 "Be well!" 的指针。

由于 p[2] 是数组元素，*++pp 是左值，所以还能对它做 --。

③ --*++pp：把 p[2] 的内容减 1，改成 c+0

```
cout << (*--*++pp+4);
```

累计输出：You



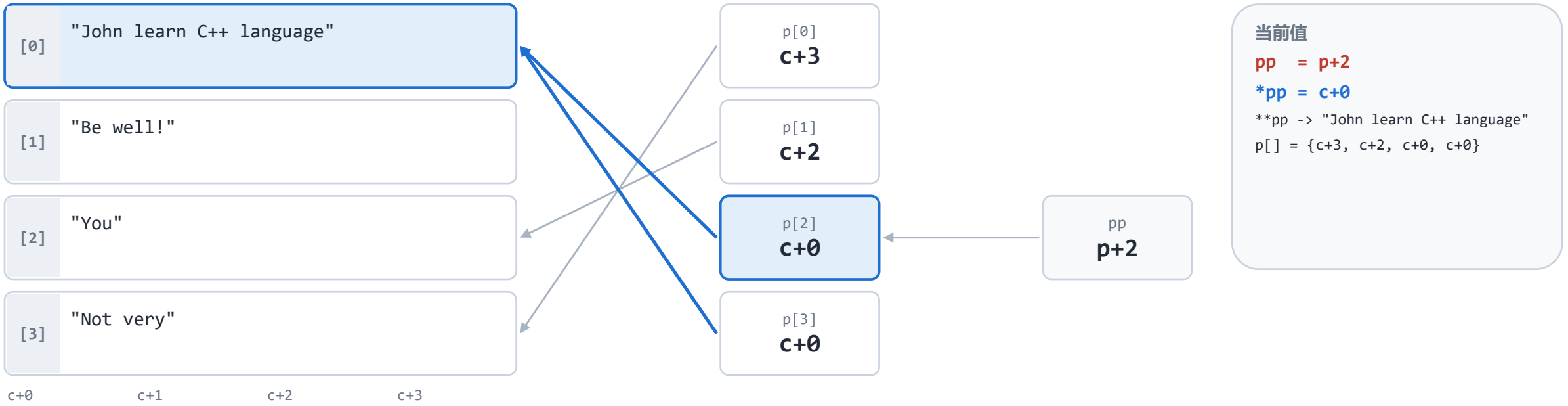
-- 作用在 p[2] 这个左值上：p[2] 由 c+1 变为 c+0。

★ 这是全题唯一的写入操作，p[2] 被永久改成 c+0，之后不再恢复。

④ 外层 * 得首地址，+4 得子串

```
cout << (*--*++pp+4);
```

累计输出：You learn C++ language



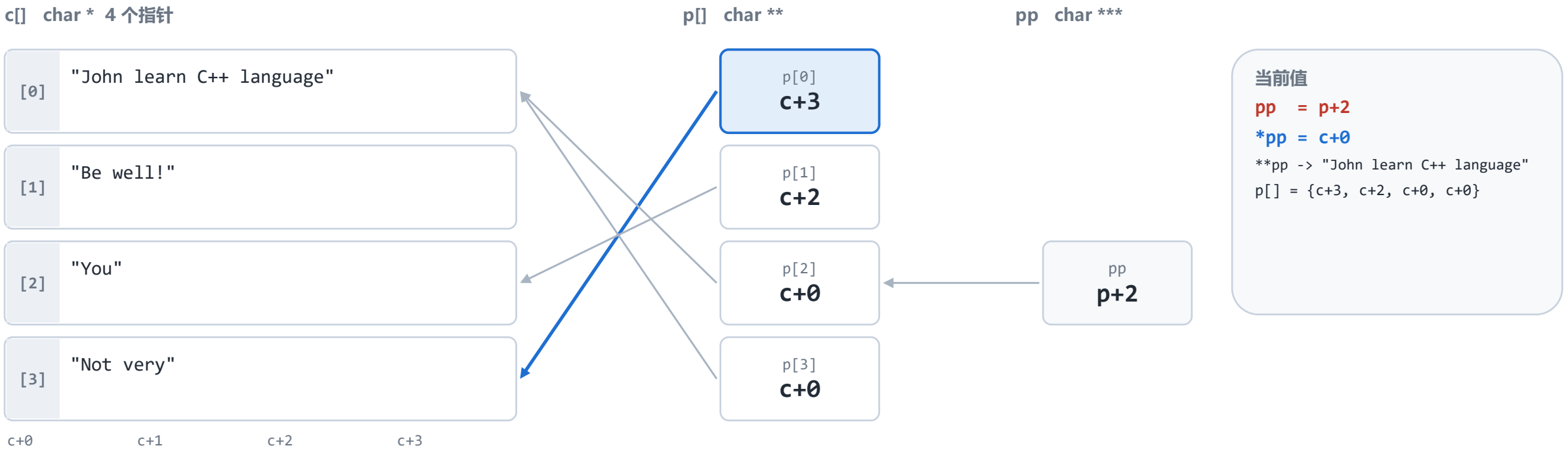
--*++pp 的结果是 c+0；外层 * 取到 c[0] 的首地址，再 +4 指向下标 4 的字符。

"John learn C++ language" + 4 -> " learn C++ language"，从空格开始输出。

① pp[-2] 等价于 *(pp-2)：退到 p[0]

```
cout << (*pp[-2]+3);
```

累计输出：You learn C++ language



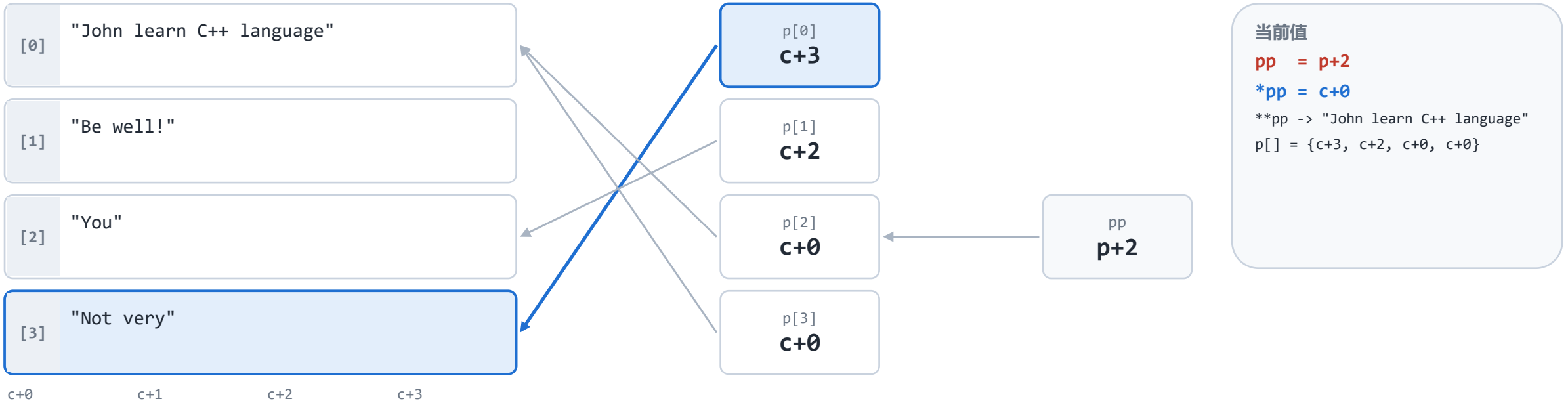
下标运算 pp[-2] 就是 *(pp-2)：从 p+2 往回退 2 个元素，落到 p[0]。

pp 本身不变，仍是 p+2；变化的只是本次取址的位置。

② 解引用得 p[0] 的值 c+3

```
cout << (*pp[-2]+3);
```

累计输出：You learn C++ language



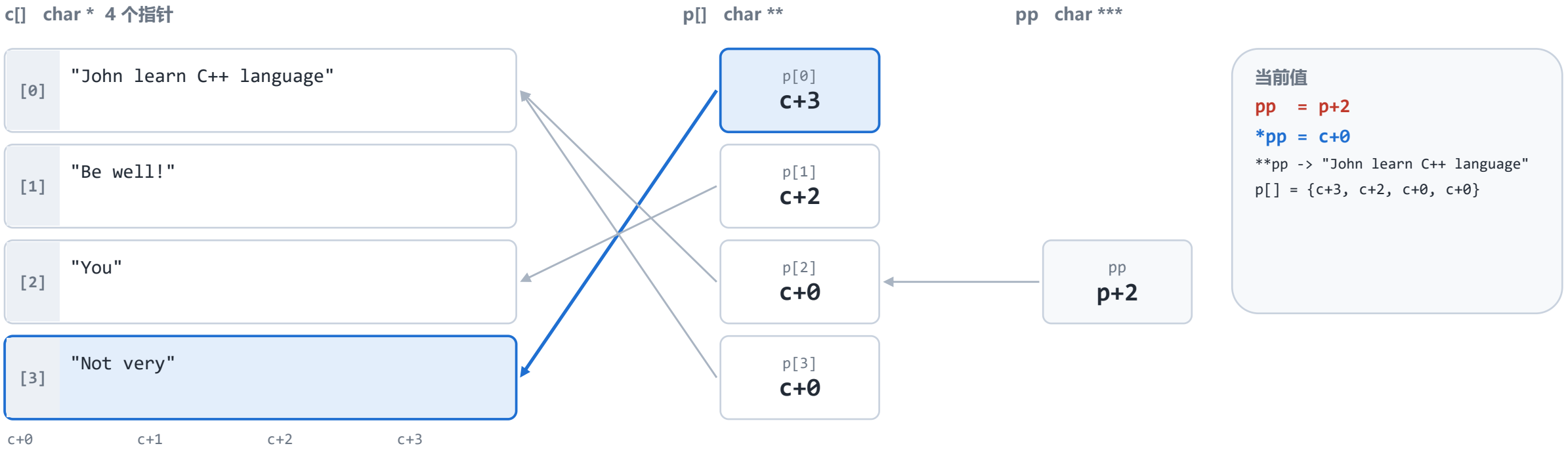
到这里得到的还是地址，尚未输出。

*（pp-2）= p[0] = c+3，指向 "Not very"。

③ +3 取子串： very

```
cout << (*pp[-2]+3);
```

累计输出：You learn C++ language very



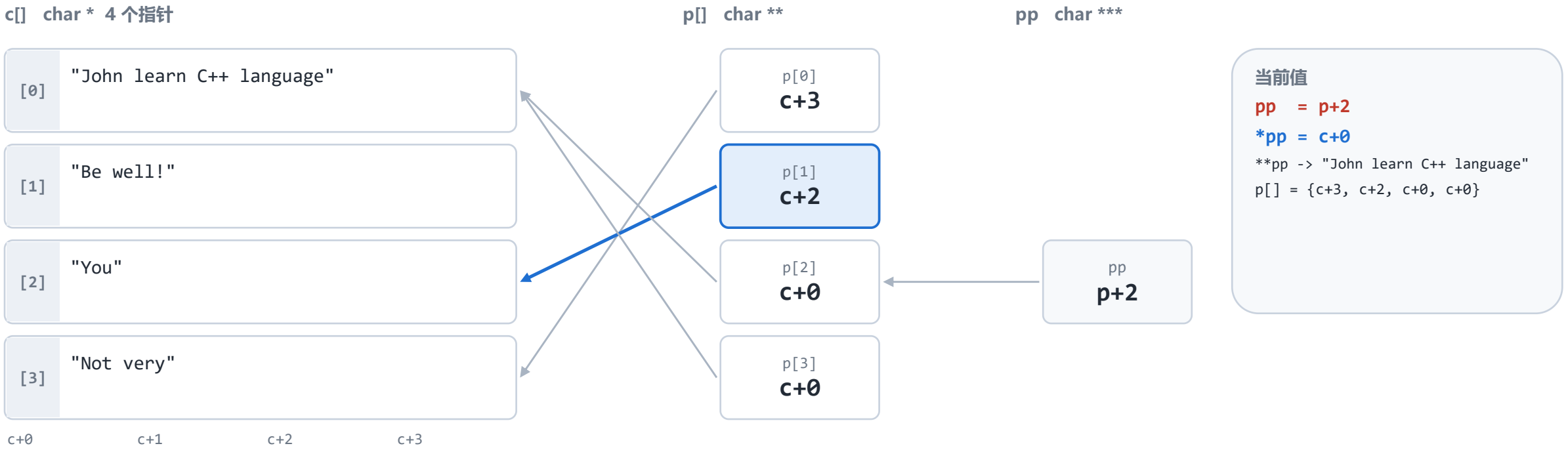
"Not very" + 3 -> " very"（下标 3 正好是那个空格）。

输出 " very"，与前文拼成 “...language very”。

① pp[-1] 等价于 *(pp-1)：落到 p[1]

```
cout << (pp[-1][-1]+2);
```

累计输出：You learn C++ language very



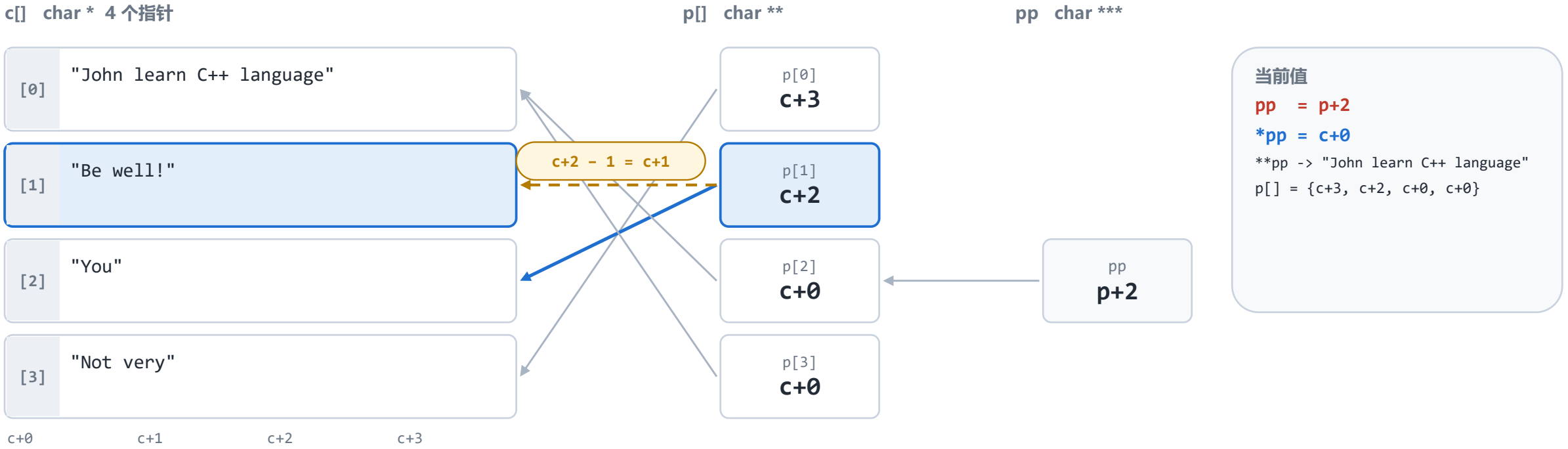
pp-1 = p+1，故 pp[-1] 是 p[1]，其值为 c+2，指向 "You"。

二维下标从左结合：先算出 pp[-1]，再对结果做 [-1]。

② pp[-1][-1]：char * 再往回退 1 个元素

```
cout << (pp[-1][-1]+2);
```

累计输出：You learn C++ language very



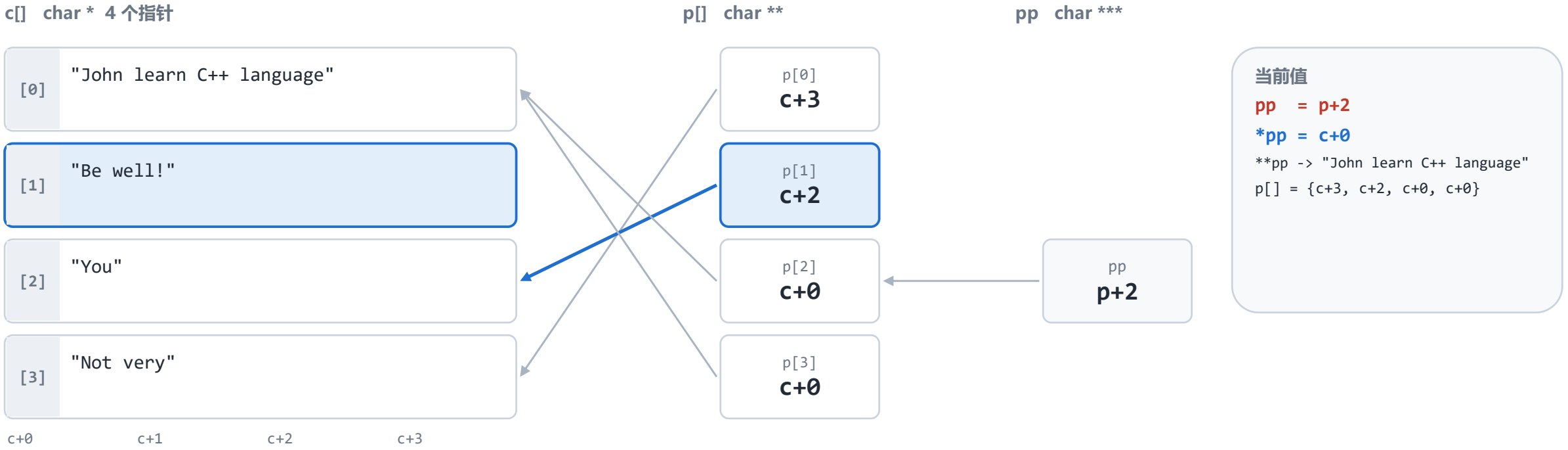
pp[-1][-1] = *(pp[-1]-1) = *(p[1]-1) = *(c+2-1) = *(c+1) = c[1]。

★ 指针减法按“元素”退：c+2 退 1 个 char * 元素，正好是 c[1]，指向 "Be well!"。

③ +2 取子串： well! 输出结束

```
cout << (pp[-1][-1]+2);
```

累计输出：You learn C++ language very well!



"Be well!" + 2 -> " well!"。四条语句全部输出完毕。

随后 cout << endl; 换行，程序结束。

运行结果与关键规律

Result

程序输出

```
You learn C++ language very well!
```

语句	输出片段	pp	p[] 变化
**++pp	You	p+1	{c+3,c+2,c+1,c}
--++pp+4	learn C++ language	p+2	{c+3,c+2,c+0,c}
*pp[-2]+3	very	p+2	不变
pp[-1][-1]+2	well!	p+2	不变

三级指针的读法

pp 是 char ***：*pp 得 char **，**pp 得 char *，***pp 才是字符。每多一层 * 就往下走一级。

自增自减落在谁身上

++pp 移动的是 pp 本身（在 p 数组里换位置）；--*++pp 改的是 p[2] 里存的地址，会写坏数组。

下标与指针等价

pp[-2] 等价于 *(pp-2)；pp[-1][-1] 等价于 *(*(pp-1)-1)。负下标合法，因为指针可以往回退。

易错点：第 2 条语句中 --*++pp 是把 p[2] 的内容由 c+1 改为 c+0，属于对数组的写入；若误写成 --pp，那只是移动 pp 自身，结果会完全不同。